

MINI PROJECT REPORT

ON

INVENTORY MANAGEMENT SYSTEM

OF

PYTHON PROGRAMMING

SUBMITTED BY :

NAME : NITISH

UID : 24BCA10598

SECTION : 24BCA -1(B)

SEMESTER - 5TH

SUBMITTED TO : MS. JYOTI

INDEX

This report follows the organization of the supplied Python mini-project format. It introduces the Inventory Management System, explains its design and implementation, presents the main program code, and provides representative colored application mockups and a repository section. The sample implementation is an educational prototype using Python, Flask, and SQLite.

S. No.	Section	Coverage
1	Abstract	Purpose, approach, and expected value
2	Project Design / Implementation	Introduction, objectives, scope, requirements, architecture, modules, database design, workflow, validation, and testing
3	Python Code	Database setup, routes, inventory operations, and reporting
4	Output / Screenshots	Representative colored pages and sample records
5	GitHub Repository Link	Repository publishing guidance
6	Conclusion and Future Scope	Outcome, limitations, and possible enhancements

Project summary

The project maintains product records, stock quantities, unit prices, suppliers, and stock movements. A manager can add products, inspect inventory, update details, record stock received, record stock issued, and review low-stock items. Data is stored in SQLite so it remains available after the application is restarted.

The implementation is a learning prototype. It demonstrates common inventory operations and validation rules; it is not a production-certified accounting or warehouse platform.

1. ABSTRACT

The Inventory Management System is a web-based application designed to help a shop, department, laboratory, or small business maintain accurate information about items held in stock. Manual stock registers and disconnected spreadsheets can become difficult to maintain when the number of products increases. They may lead to duplicated entries, incorrect quantities, delayed reordering, and difficulty identifying which products are running low.

This project uses Python as the primary programming language, Flask as the web framework, HTML and CSS for the user interface, and SQLite as the database. The application organizes product details such as product code, name, category, quantity, unit price, supplier, and reorder level. It supports basic stock-in and stock-out transactions. Before reducing stock, the system checks that the requested quantity is available.

The system is structured around practical modules: dashboard, product registration, inventory listing, product update, stock movement, low-stock monitoring, and summary reporting. Flask routes receive form submissions, validate data, execute database operations, and return updated pages.

The main objective is to demonstrate how a Python application can simplify routine inventory work. The prototype improves visibility into current stock and provides a foundation for user authentication, barcode scanning, purchase orders, supplier management, CSV export, audit logs, and secure deployment.

Keywords

Inventory, stock control, Python, Flask, SQLite, product management, stock-in, stock-out, reorder level, CRUD, database application.

2. PROJECT DESIGN / IMPLEMENTATION

2.1 Introduction

Inventory refers to the goods, materials, spare parts, or supplies an organization keeps for use or sale. Inventory management is the process of recording these items, monitoring their quantities, and ensuring that the right amount of stock is available at the right time. A small store may hold hundreds of products, each with a different purchase price, selling price, supplier, and minimum acceptable quantity.

A manual method generally relies on paper registers or spreadsheets. These methods become less reliable as transactions increase. A product may be issued without the register being updated, a new delivery may be entered twice, or a reorder may be delayed because no one notices the remaining quantity. These issues can affect customer service and increase operating costs.

The proposed application centralizes product information in one database. A user can open the system through a browser, select an operation, enter the required data, and receive a confirmation. The application applies validation rules before saving changes.

2.2 Problem statement

The problem addressed by this project is the difficulty of maintaining consistent stock records using manual or scattered methods. The system aims to reduce repetitive data entry, make current quantities easier to inspect, and identify products that need replenishment.

2.3 Proposed solution

The solution is a Flask web application with an SQLite database. Product records are stored in a products table, while stock movements are recorded in a separate transactions table. Keeping movements separate from the current quantity helps create a basic history of stock changes.

2.4 Objectives

- Centralized product records for product code, name, category, quantity, prices, suppliers, and reorder levels.
- Faster stock updates through stock-in and stock-out forms.
- Quantity validation to reject negative values and prevent stock-out beyond available stock.
- Low-stock awareness by comparing current quantity with reorder level.
- Persistent storage using SQLite.
- Clear interface with tables, forms, and status messages.
- Learning outcomes in Python, Flask routes, SQL, HTML forms, and testing.

2.5 Project scope

The prototype covers a single inventory location and a basic manager workflow. It can add, view, edit, and delete products; record stock movements; show current quantities; and list products at or below reorder levels. Multi-warehouse transfers, advanced accounting, invoices, barcode devices, cloud synchronization, and enterprise integrations are outside the current scope.

2.6 Benefits

Centralized records improve consistency and reduce the effort required to locate product information. Low-stock indicators help users prioritize replenishment. Stock movement history makes it easier to explain why a quantity changed. Automated calculations reduce arithmetic errors, while modular design makes the application easier to maintain.

2.7 Technologies Used

Technology	Role	Reason for selection
Python 3	Application logic and validation	Readable syntax and broad ecosystem
Flask	Routing and request handling	Lightweight web framework
SQLite	Persistent product and transaction storage	Serverless relational database
HTML5	Forms, tables, page structure	Standard browser markup
CSS3	Layout, spacing, colors	Separates visual styling from logic
Jinja2	Dynamic HTML templates	Displays database records in pages
VS Code / IDE	Editing and debugging	Convenient development environment
Browser	Access to the application	No separate desktop client required

Development environment

A recommended environment is Python 3.10 or later, Flask installed in a virtual environment, a code editor, and a modern browser. Run locally during development. Before public deployment, disable debug mode, configure secrets securely, and add access controls.

2.8 System Requirements

Hardware requirements

- Modern dual-core processor or equivalent.
- 4 GB RAM recommended.
- Approximately 100 MB for source code, environment, and a small database; more space may be needed forbackups.
- Laptop or desktop display capable of showing a browser.

Software requirements

- Windows, Linux, or macOS.
- Python 3 and pip.
- Flask package.
- Code editor or Python IDE.
- Modern web browser.

Functional requirements

ID	Requirement	Expected behavior
FR-01	Add product	Save valid details and a unique product code
FR-02	View inventory	Display products, quantities, and prices
FR-03	Update product	Modify details of an existing product
FR-04	Delete product	Remove after a deliberate action
FR-05	Stock-in	Increase quantity and record movement
FR-06	Stock-out	Decrease only when enough stock exists
FR-07	Low-stock list	Show items at or below reorder level

FR-08	Summary	Show product count and total units

2.9 Non-Functional Requirements

Usability

Forms should use descriptive labels, mark required fields, and display understandable feedback. Tables should use clear column headings and provide simple navigation.

Reliability and data integrity

Validate data before committing it. Product codes should be unique, quantities should be whole numbers, prices should not be negative, and stock-out operations should not exceed available stock.

Security

A deployed system needs authenticated users, role-based permissions, CSRF protection, secure session settings, secret-key management, and parameterized SQL queries. Passwords must never be stored in plain text.

Feasibility study

Type	Assessment
Technical	Python, Flask, and SQLite are sufficient for the prototype.
Economic	Core tools are free; main cost is development time and optional hosting.
Operational	Browser-based interface is familiar and simple.
Schedule	The feature set is suitable for a semester mini project.

2.10 System Architecture

The application follows a simple three-layer arrangement. The presentation layer contains browser pages and forms. The application layer contains Flask routes, validation, and inventory rules. The data layer stores products and transaction history in SQLite.

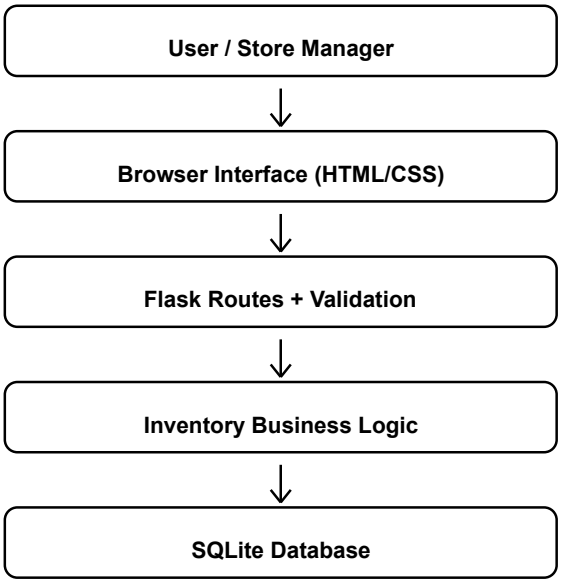


Figure 1 — High-level architecture of the Inventory Management System.

Data flow

- User selects an operation and submits a form.
- Flask validates the request.
- Application reads or updates SQLite.
- Valid changes are committed and a refreshed page or confirmation is returned.

2.11 Main Modules

Dashboard

Shows product count, total units in stock, and number of products at or below reorder level.

Product management

Registers product code, name, category, quantity, unit price, supplier, and reorder level. Product code is a unique identifier.

Inventory listing and search

Displays products in a table and can be extended to search by product code, name, or category.

Stock-in

Represents goods received or returned. The system checks for a positive quantity, increases stock, and records an IN transaction.

Stock-out

Represents goods issued, sold, consumed, or removed. The system verifies that sufficient units are available before reducing stock.

Low-stock monitoring

Selects products where current quantity is less than or equal to the reorder level.

2.12 System Workflow and Algorithm

Product creation algorithm

```
START Receive product code, name, category, quantity,
price, supplier, and reorder level Validate required
fields and numeric values
Check whether product code already exists
IF code exists: display duplicate-code message
ELSE: insert product, commit, and display success
END IF
END
```

Stock movement algorithm

```
START
Receive product ID, movement type, and quantity
Validate movement type and positive quantity
Find product
IF product not found: display error
ELSE IF OUT quantity exceeds current stock: reject
ELSE: update quantity and insert transaction record
Commit both changes together
END IF
END
```

Key business rules

- Product code must be unique.
- Stock quantity cannot be negative.
- Movement quantity must be positive.
- Stock-out cannot exceed available stock.
- Low-stock threshold is a warning, not an automatic purchase.
- Every successful movement should create a transaction record.

2.13 Database Design

SQLite stores data in inventory.db. The products table contains current product information; transactions stores each successful stock movement and links it to a product through product_id.

Table: products

Field	Type	Description
id	INTEGER	Primary key
product_code	TEXT	Unique product identifier
name	TEXT	Required product name
category	TEXT	Product category
quantity	INTEGER	Current stock, zero or greater
unit_price	REAL	Price, zero or greater
supplier	TEXT	Supplier or source
reorder_level	INTEGER	Low-stock threshold
created_at	TEXT	Creation timestamp

Table: transactions

Field	Type	Description
id	INTEGER	Primary key
product_id	INTEGER	Reference to products.id
movement_type	TEXT	IN or OUT
quantity	INTEGER	Positive quantity moved

note	TEXT	Optional reason
created_at	TEXT	Movement timestamp

2.14 Entity Relationship Model

The relationship between products and transactions is one-to-many: one product can have many stock movement records, while each transaction belongs to one product.

PRODUCTS (1) ■■■■■■■■■■< (M) TRANSACTIONS

Database creation example

```
CREATE TABLE products (   id INTEGER PRIMARY KEY AUTOINCREMENT,
product_code TEXT NOT NULL UNIQUE,   name TEXT NOT NULL,   category
TEXT NOT NULL DEFAULT '',   quantity INTEGER NOT NULL DEFAULT 0
CHECK(quantity >= 0),   unit_price REAL NOT NULL DEFAULT 0
CHECK(unit_price >= 0),   supplier TEXT NOT NULL DEFAULT '',
reorder_level INTEGER NOT NULL DEFAULT 5 CHECK(reorder_level >= 0),
created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP );

CREATE TABLE transactions (   id INTEGER PRIMARY KEY AUTOINCREMENT,
product_id INTEGER NOT NULL,   movement_type TEXT NOT NULL
CHECK(movement_type IN ('IN','OUT')),   quantity INTEGER NOT NULL
CHECK(quantity > 0),   note TEXT NOT NULL DEFAULT '',
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY(product_id) REFERENCES products(id)
);
```

Low-stock query

```
SELECT product_code, name, quantity, reorder_level
FROM products
WHERE quantity <= reorder_level
ORDER BY quantity ASC, name ASC;
```

2.15 User Interface Design

The interface is task-focused. The home page contains a title, summary cards, navigation, and a low-stock list. Product forms group related fields and use numeric controls for quantities and prices. The inventory table makes product identity, current quantity, and reorder level easy to compare.

Page	Main controls	Purpose
Dashboard	Summary cards and navigation	At-a-glance status
Add Product	Code, name, category, quantity, price, supplier, reorder level	Register a product
Inventory	Search, table, edit/delete actions	Manage product records
Stock Movement	Product, IN/OUT, quantity, note	Record stock change
History	Date, product, type, quantity, note	Review movements

Usability principles

• Consistent navigation.

• Clear success and error messages.

• Visible field labels.

• Readable contrast and spacing.

• Confirmation for destructive actions.

• Low-stock warnings use a label as well as color.

2.16 Implementation Procedure

Step 1: Install Flask

```
python -m venv venv # Windows:
venv\Scripts\activate # macOS/Linux:
source venv/bin/activate pip install
flask
```

Step 2: Project structure

```
inventory_management/
app.py
inventory.db
templates/
static/
style.css
```

A compact classroom version can begin in app.py. A maintainable version moves HTML into a templates folder and CSS into static/style.css.

Step 3: Initialize database

Create tables with CREATE TABLE IF NOT EXISTS. Initialization should not delete existing records.

Step 4: Run and verify

```
python app.py # Open
http://127.0.0.1:5000
```

Test valid and invalid form values. Confirm that saved products remain after refresh, stock movements update quantities, and rejected stock-out requests leave the database unchanged.

3. PYTHON CODE

The following core code illustrates a Flask and SQLite implementation. The application should be tested in the student's own environment before submission.

3.1 Imports and database

```
from flask import Flask, request, redirect, url_for, render_template_string
import sqlite3 from contextlib import closing

app = Flask(__name__)
app.secret_key = "replace-with-a-random-secret" DATABASE
= "inventory.db"

def get_db():
    conn = sqlite3.connect(DATABASE)
conn.row_factory = sqlite3.Row
conn.execute("PRAGMA foreign_keys = ON")
return conn

def init_db():     with
closing(get_db()) as conn:
        conn.executescript("""         CREATE
TABLE IF NOT EXISTS products (             id
INTEGER PRIMARY KEY AUTOINCREMENT,
product_code TEXT NOT NULL UNIQUE,
name TEXT NOT NULL,             category TEXT NOT
NULL DEFAULT '',
            quantity INTEGER NOT NULL DEFAULT 0 CHECK(quantity >= 0),
unit_price REAL NOT NULL DEFAULT 0 CHECK(unit_price >= 0),             supplier
TEXT NOT NULL DEFAULT '',             reorder_level INTEGER NOT NULL DEFAULT 5
CHECK(reorder_level >= 0),             created_at TEXT NOT NULL DEFAULT
CURRENT_TIMESTAMP         );         CREATE TABLE IF NOT EXISTS transactions (
id INTEGER PRIMARY KEY AUTOINCREMENT,             product_id INTEGER NOT NULL,
movement_type TEXT NOT NULL CHECK(movement_type IN ('IN','OUT')),
quantity INTEGER NOT NULL CHECK(quantity > 0),             note TEXT NOT NULL
DEFAULT '',
            created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY(product_id) REFERENCES products(id)
        );
""")
conn.commit()

init_db()
```

The database helper returns rows that can be accessed by column name. The foreign-key pragma is enabled for each connection. Replace the placeholder secret with a secure environment-based value before deployment.

3.2 Validation and dashboard

```
def as_nonnegative_int(value, field_name):
try:
        number = int(value)
except (TypeError, ValueError):
        raise ValueError(f"{field_name} must be a whole number.")
if number < 0:          raise ValueError(f"{field_name} cannot be
negative.")     return number

def as_price(value):
try:
        price = float(value)
except (TypeError, ValueError):
        raise ValueError("Unit price must be numeric.")
if price < 0:          raise ValueError("Unit price cannot
be negative.")     return price

@app.route("/") def dashboard():     with closing(get_db()) as conn:
product_count = conn.execute(              "SELECT COUNT(*) FROM
products").fetchone()[0]         total_units = conn.execute(
"SELECT COALESCE(SUM(quantity), 0) FROM products").fetchone()[0]
low_stock = conn.execute(             """SELECT * FROM products
            WHERE quantity <= reorder_level
ORDER BY quantity, name""").fetchall()     return
render_template_string(DASHBOARD_HTML,
product_count=product_count,
        total_units=total_units, low_stock=low_stock)

DASHBOARD_HTML = """
<!doctype html><title>Inventory Dashboard</title>
<h1>Inventory Management System</h1>
<p>Products: {{ product_count }}</p>
<p>Total units in stock: {{ total_units }}</p>
<p><a href="/products">View inventory</a> |
<a href="/products/add">Add product</a> |
<a href="/movement">Record stock movement</a></p>
<h2>Low-stock items</h2>
{% for p in low_stock %}
<p>{{ p.product_code }} — {{ p.name }}: {{ p.quantity }}
( reorder level {{ p.reorder_level }} )</p>
{% else %}<p>No low-stock products.</p>{% endfor %}
"""
```

Validation converts form values into expected types. The dashboard uses aggregate queries and a reorder-level condition to display a summary.

3.3 Add and list products

```
PRODUCTS_HTML = """
<!doctype html><title>Inventory</title><h1>Product Inventory</h1>
<p><a href="/">Dashboard</a> | <a href="/products/add">Add product</a></p>
{% for p in products %}
<p>{{ p.product_code }} | {{ p.name }} | {{ p.category }} |
Qty: {{ p.quantity }} | Price: {{ '%.2f'|format(p.unit_price) }}</p>
{% endfor %}
"""

@app.route("/products") def
products():     with
closing(get_db()) as conn:
```

```
        rows = conn.execute(             "SELECT * FROM products
ORDER BY name").fetchall()     return
render_template_string(PRODUCTS_HTML, products=rows)

PRODUCT_FORM = """
<!doctype html><title>Add product</title><h1>Add a Product</h1>
<form method="post">
<label>Product code <input name="product_code" required></label><br>
<label>Name <input name="name" required></label><br>
<label>Category <input name="category"></label><br>
<label>Opening quantity <input name="quantity" type="number" min="0" value="0" required></label><br>
<label>Unit price <input name="unit_price" type="number" min="0" step="0.01" value="0" required></
label><br> <label>Supplier <input name="supplier"></label><br> <label>Reorder level <input
name="reorder_level" type="number" min="0" value="5" required></ label><br> <button
type="submit">Save product</button></form>
"""

@app.route("/products/add", methods=["GET", "POST"])
def add_product():     if request.method == "GET":
        return render_template_string(PRODUCT_FORM)
code = request.form.get("product_code", "").strip()
name = request.form.get("name", "").strip()
category = request.form.get("category", "").strip()
supplier = request.form.get("supplier", "").strip()
if not code or not name:
        return "Product code and name are required.", 400
try:
        qty = as_nonnegative_int(request.form.get("quantity"), "Quantity")
reorder = as_nonnegative_int(request.form.get("reorder_level"), "Reorder level")
price = as_price(request.form.get("unit_price"))         with closing(get_db()) as conn:
            conn.execute("""INSERT INTO products
(product_code,name,category,quantity,unit_price,supplier,reorder_level)
                VALUES (?,?,?,?,?,?,?)""",
(code,name,category,qty,price,supplier,reorder))
conn.commit()     except ValueError as error:         return
str(error), 400     except sqlite3.IntegrityError:
        return "Product code already exists.", 409
return redirect(url_for("products"))
```

The route validates required fields and numeric values, and uses placeholders in the SQL insert. The database UNIQUE constraint rejects duplicate product codes.

3.4 Stock movement

```
MOVEMENT_FORM = """
<!doctype html><title>Stock movement</title><h1>Record Stock Movement</h1>
<form method="post">
<label>Product ID <input name="product_id" type="number" min="1" required></label><br>
<label>Movement <select name="movement_type">
<option value="IN">Stock In</option><option value="OUT">Stock Out</option>
</select></label><br>
<label>Quantity <input name="quantity" type="number" min="1" required></label><br>
<label>Note <input name="note"></label><br><button>Save movement</button></form>
"""

@app.route("/movement", methods=["GET", "POST"])
def movement():     if request.method == "GET":
        return render_template_string(MOVEMENT_FORM)
try:
```

```
        product_id = int(request.form.get("product_id", ""))
qty = int(request.form.get("quantity", ""))     except
ValueError:
        return "Product ID and quantity must be whole numbers.", 400
kind = request.form.get("movement_type", "")     note =
request.form.get("note", "").strip()     if product_id < 1 or qty < 1
or kind not in ("IN", "OUT"):         return "Invalid movement
details.", 400

    with closing(get_db()) as conn:
try:
            conn.execute("BEGIN IMMEDIATE")             product =
conn.execute(                "SELECT quantity FROM products WHERE id = ?",
(product_id,)).fetchone()             if product is None:
conn.rollback()                 return "Product not found.", 404
current = product["quantity"]             if kind == "OUT" and qty > current:
conn.rollback()                 return "Insufficient stock.", 400
new_qty = current + qty if kind == "IN" else current - qty
conn.execute("UPDATE products SET quantity=? WHERE id=?",
(new_qty, product_id))             conn.execute("""INSERT INTO transactions
(product_id,movement_type,quantity,note) VALUES (?,?,?,?)""",
(product_id,kind,qty,note))             conn.commit()         except
Exception:             conn.rollback()             raise
    return redirect(url_for("products"))
```

The quantity update and history insert are committed together. A stock-out request is rejected if it exceeds current quantity.

3.5 Transaction history and application start

```
@app.route("/transactions") def
transactions():     with
closing(get_db()) as conn:
        rows = conn.execute("""SELECT t.created_at, p.name AS product_name,
            t.movement_type, t.quantity, t.note             FROM
transactions t JOIN products p ON p.id=t.product_id             ORDER
BY t.id DESC""").fetchall()     return render_template_string("""
<!doctype html><title>Transaction History</title>
    <h1>Stock Transaction History</h1>
    {% for t in rows %}
    <p>{{ t.created_at }} | {{ t.product_name }} |
       {{ t.movement_type }} | {{ t.quantity }} | {{ t.note }}</p>
{% endfor %}""", rows=rows)

if __name__ == "__main__":     # Debug mode is
for local development only.
app.run(debug=True)
```

How to run

- Save the code as app.py.
- Install Flask with pip install flask.
- Run python app.py.
- Open http://127.0.0.1:5000.
- Add a product and record stock movements.
- Check inventory and transaction history.

The code is a classroom prototype. For a final polished version, move HTML into separate templates, add CSS, authentication, CSRF protection, and production deployment settings.

3. TESTING AND VALIDATION

Testing checks whether the application behaves according to its requirements. Tests should cover normal operations, invalid input, and boundary cases.

Test ID	Test case	Expected result
TC-01	Add valid product	Saved and visible in inventory
TC-02	Duplicate product code	Rejected; no duplicate record
TC-03	Blank product name	Validation error
TC-04	Negative quantity	Request rejected
TC-05	Non-numeric price	Request rejected
TC-06	Stock-in 10 units	Quantity increases by 10 and IN record saved
TC-07	Stock-out within available stock	Quantity decreases and OUT record saved
TC-08	Stock-out beyond available stock	Rejected; quantity unchanged
TC-09	Low-stock query	Products at/below reorder level shown
TC-10	Restart application	Saved records remain
TC-11	View history	Movements appear newest first

Testing approach

Unit tests can verify validation helpers and quantity calculations. Integration tests can use a temporary database to check the full route-and-database flow. Manual UI tests can confirm navigation and feedback. This is a test plan; actual results should be recorded after running the application.

5. OUTPUT / SCREENSHOTS

The following are colored illustrative mockups of application pages. They demonstrate the intended visual appearance; they are not screenshots from a verified running application. Replace them with actual screenshots after running and testing the code.

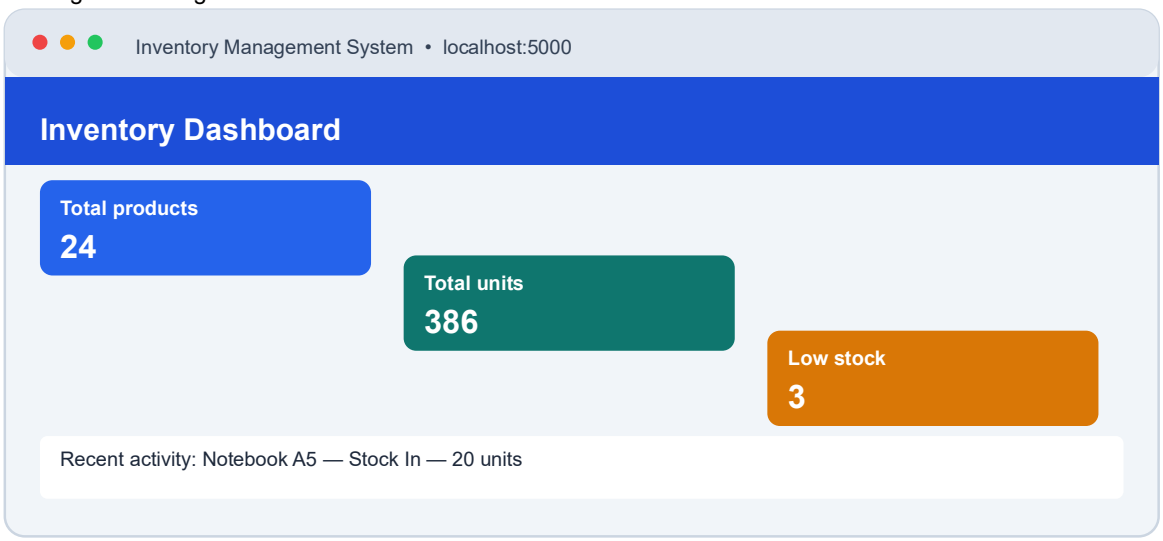


Figure 2 — Colored dashboard mockup (sample data).

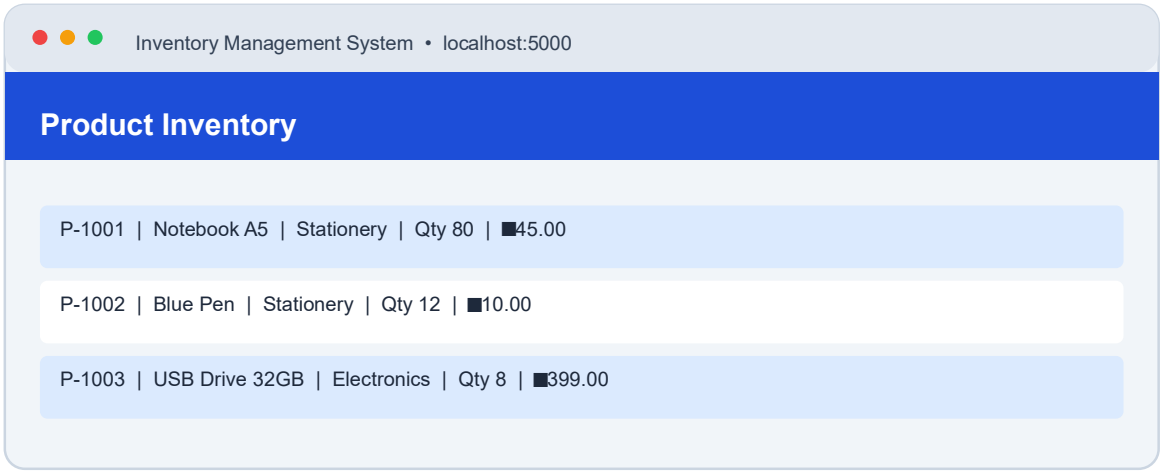


Figure 3 — Colored inventory list mockup (sample data).

5.1 Additional Output Examples

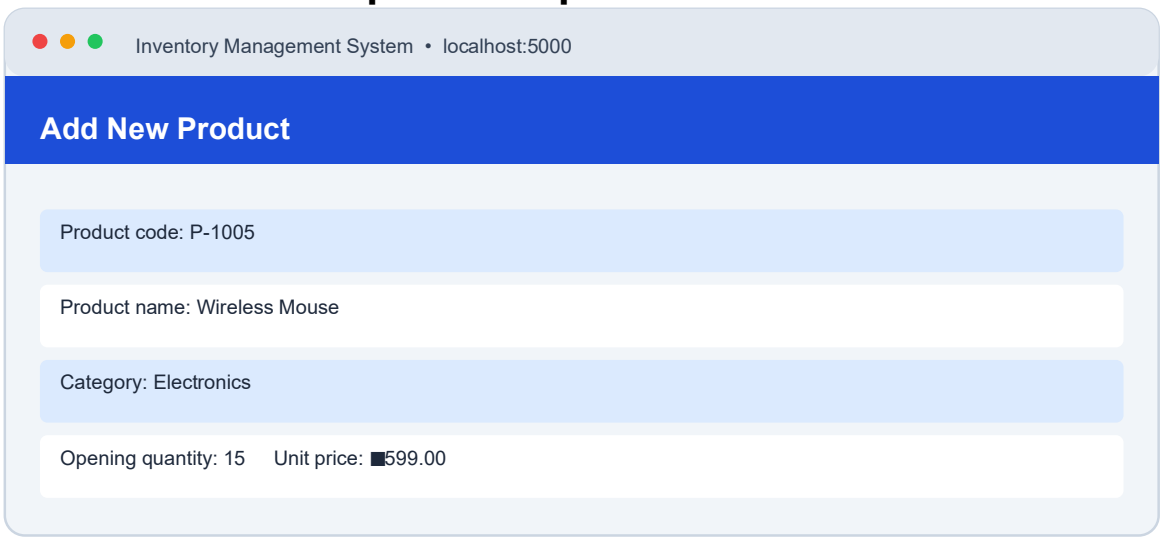


Figure 4 — Product form mockup. Values are illustrative.

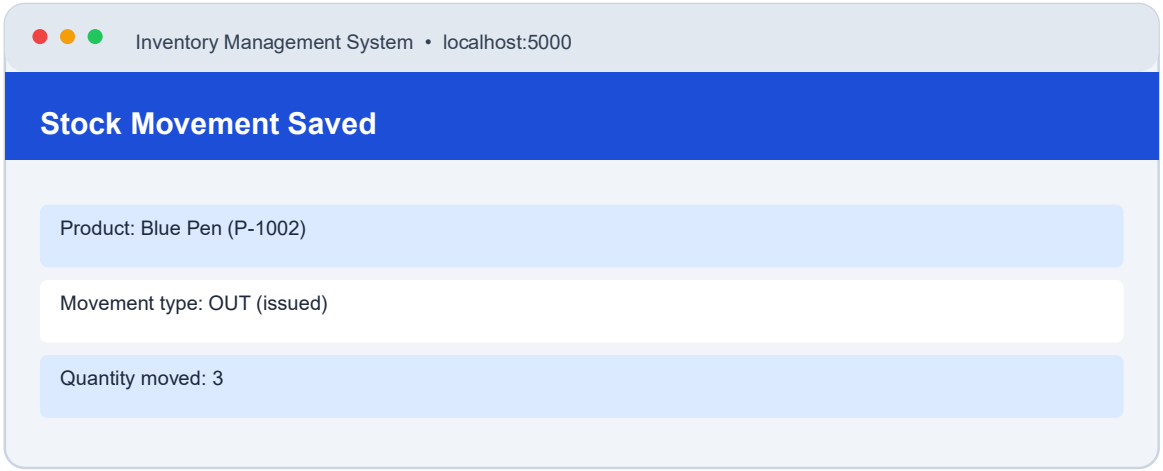


Figure 5 — Stock-out result mockup. Sample values only.

5.2 Low-Stock Report and Transaction History

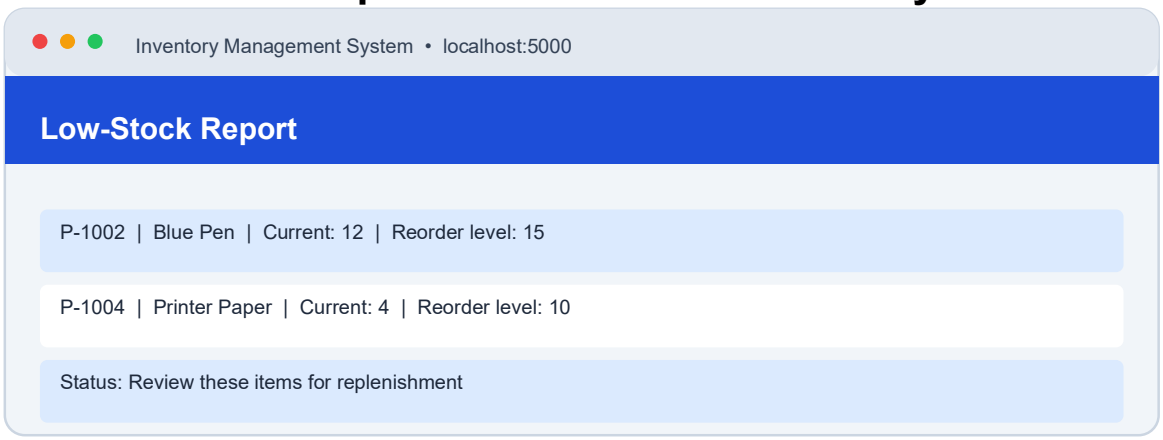


Figure 6 — Low-stock report mockup.

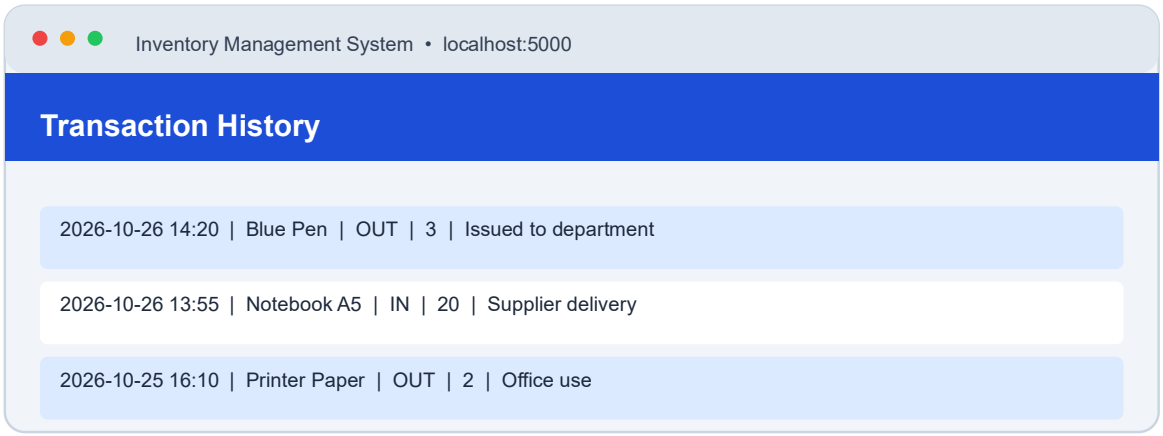


Figure 7 — Transaction history mockup. Replace with actual application captures before submission.

6. RESULTS, LIMITATIONS AND FUTURE SCOPE

6.1 Expected results

When implemented and tested, the system should provide a central inventory list, support product creation, validate stock movements, and display low-stock items. Stock-in increases the quantity, while a valid stock-out decreases it.

Successful movements are stored in the transaction table.

6.2 Limitations

• One local inventory location only.

• Authentication and role-based authorization are not fully implemented.

• Minimal pages need additional CSS for a polished experience.

• No purchase orders, invoices, supplier payment tracking, or tax calculations.

• No scheduled backup, protected audit log, or deployment hardening.

• Output figures and mockups are illustrative, not actual test results.

6.3 Future enhancements

• Secure login, password hashing, roles, and audit trails.

• Supplier and purchase-order modules.

• Barcode or QR-code scanning.

• CSV/PDF exports and report filters.

• Low-stock notifications.

• Multiple warehouses and stock transfers.

• Unit/integration tests and automated regression testing.

• HTTPS deployment, backups, and monitoring.

7. CONCLUSION

The Inventory Management System is a practical Python mini project that demonstrates how a web application can organize stock records and support common inventory tasks. Flask handles requests and application flow, while SQLite stores product information and stock movement history. Validation rules help prevent duplicate product codes, invalid quantities, negative prices, and stock-out operations that exceed available stock.

The project brings together programming, database management, web forms, and basic software engineering. It provides a starting point for a small store or department, but it should be extended and security-tested before real business use. Actual testing and screenshots should be completed in the local development environment before final submission.

References

• Python Software Foundation. Python Documentation — https://docs.python.org/3/

• Pallets Projects. Flask Documentation — https://flask.palletsprojects.com/

• SQLite Consortium. SQLite Documentation — https://www.sqlite.org/docs.html

• MDN Web Docs. HTML — https://developer.mozilla.org/docs/Web/HTML

• MDN Web Docs. CSS — https://developer.mozilla.org/docs/Web/CSS

THANK YOU